# UNIVERSITY INSTITUTE OF COMPUTING

Bachelors of Computer Applications/Science

# Project Report

# On

# Resume Analyzer

Submitted by

**Student Name: Vaibhav Kamboj**

**UID:24bca10646**

**Section/Group:24BCA-3A**

DISCOVER . LEARN . EMPOWER

1

## BONAFIDE CERTIFICATE

# BONAFIDE CERTIFICATE

Certified that the project report entitled "**RESUME ANALYZER** (CampusConnect AI)" is the bonafide work of the student(s) named below, carried out under the supervision of the undersigned, in partial fulfillment of the requirements for the award of the Bachelor of Computer Applications degree at Chandigarh University.

Student Name: Vaibhav Kamboj

UID: 24BCA10646

This report records the work completed for the academic project. The implementation described in this report is based on the source code and project package submitted for review.

| **Head of Department** | **Project Supervisor** |
|---|---|
| Signature: _____<br>Name: _____<br>Designation: _____<br>Department: _____ | Signature: _____<br>Name: _____<br>Designation: _____<br>Department: _____ |

Submitted for the project viva-voce examination held on: _____

| **Internal Examiner** | **External Examiner** |
|---|---|
| Signature: _____<br>Name: _____ | Signature: _____<br>Name: _____ |

*The certificate must be signed and approved by the department. Do not insert signatures without authorization.*

# DECLARATION

We hereby declare that the project entitled “Resume Analyzer (CampusConnect AI)” is an academic project carried out by us as part of the Bachelor of Computer Applications programme. The work presented in this report is based on the application source code supplied with the project and our understanding of its design and implementation.

We further declare that the report has been prepared for academic evaluation. Wherever external documentation or learning resources have been used, they are acknowledged in the References section. Any third-party libraries and frameworks used by the project remain the property of their respective authors and maintainers.

We understand that the resume-analysis score is generated by programmed rules and keyword checks in the current implementation. It should not be represented as a scientifically validated hiring prediction or as a guarantee of selection.

Student Name(s): Vaibhav Kamboj

UID: 24BCA10646

Signature:

# ACKNOWLEDGEMENT

We express our sincere gratitude to Chandigarh University and the Department of UIC for providing the academic environment and resources required to complete this project.

We are thankful to our project supervisor, Mr. Shobhit Mahajan for guidance, suggestions and encouragement during the planning, development and documentation of the Resume Analyzer. We also acknowledge the support of faculty members and classmates who provided feedback that helped us understand the practical needs of students preparing for placement opportunities.

We would like to thank the developers and documentation teams behind React, Node.js, Express, MongoDB, Mongoose, Vite, Multer and pdf-parse. Their open-source tools and learning resources made it possible to develop the application using a modern web technology stack.

Finally, we thank our family and friends for their patience and motivation throughout the project.

## TABLE OF CONTENTS


| Section | Description | Page |
|---|---|---|
| Preliminary | Cover page, certificate, declaration, acknowledgement | 1–4 |
| Abstract | Project summary | 5 |
| Graphical Abstract | System overview diagram | 6 |
| Abbreviations and Symbols | Terms and notation used | 7 |
| Chapter 1 | Introduction | 8–12 |
| Chapter 2 | Literature Review / Background Study | 13–17 |
| Chapter 3 | Design Flow / Process | 18–23 |
| Chapter 4 | Results Analysis and Validation | 24–28 |
| Chapter 5 | Conclusion and Future Work | 29–30 |
| References | Technical documentation and resources | 31 |
| Appendix | Design checklist and sample test cases | 32 |
| User Manual | Step-by-step operation guide | 33 |


*Page numbers are indicative for this prepared edition. Word may repaginate slightly if names, screenshots or additional content are inserted.*

# LIST OF FIGURES AND TABLES

## List of Figures


| Figure No. | Title | Page |
|---|---|---|
| Figure 1.1 | Project scope and user groups | 10 |
| Figure 3.1 | High-level system architecture | 19 |
| Figure 3.2 | Resume analysis workflow | 22 |
| Figure 3.3 | Resume score components | 23 |
| Figure 4.1 | Resume Analyzer result categories (conceptual) | 26 |


## List of Tables


| Table No. | Title | Page |
|---|---|---|
| Table 1.1 | Project tasks and deliverables | 11 |
| Table 1.2 | Indicative project timeline | 12 |
| Table 2.1 | Comparison of existing approaches | 15 |
| Table 3.1 | Functional requirements | 18 |
| Table 3.2 | Technology selection | 20 |
| Table 3.3 | Resume scoring weights | 23 |
| Table 4.1 | Validation test cases | 27 |
| Table 4.2 | Limitations and risks | 28 |

## ABSTRACT

Resume Analyzer is a web-based application developed as part of the **CampusConnect AI project**. It is intended to help students review the structure and content of a resume before applying for internships or placement opportunities. A resume is an important professional document, yet students may find it difficult to identify **missing sections, weak descriptions,** limited skill evidence or incomplete contact information.

The submitted implementation provides a browser-based interface built with React and Vite, together with a Node.js and Express backend. A student can upload a text-based PDF resume. The backend receives the file through Multer, extracts text using pdf-parse, and evaluates the extracted content using predefined rules, regular expressions and keyword lists. It checks contact information, profile summary, education, projects, experience, skills, certifications, achievements and online professional presence.

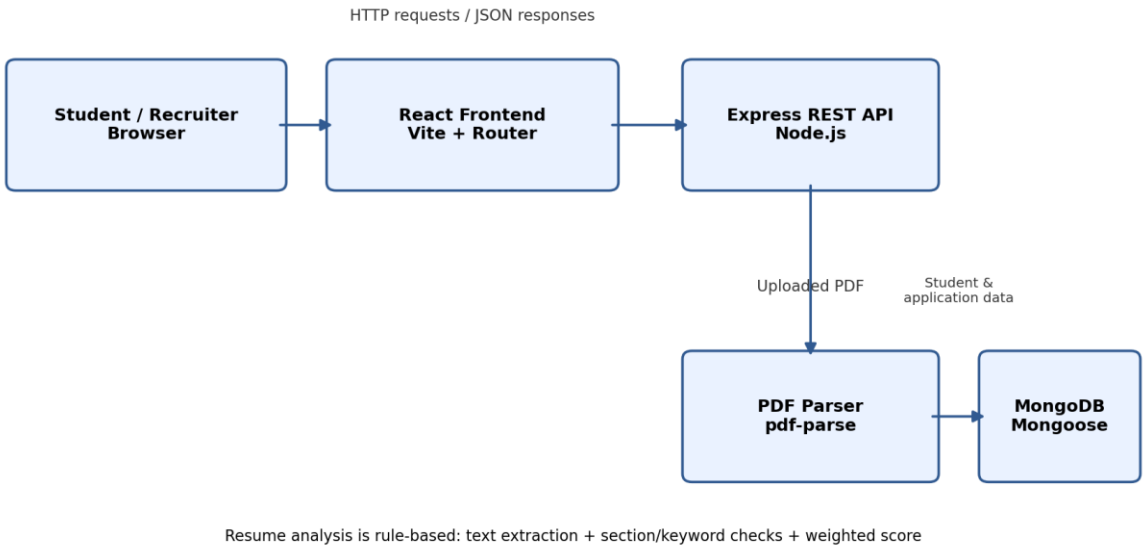
A weighted score is calculated from the presence of selected resume elements. The system also returns detected technical skills, strengths, improvement suggestions, **highlighted issues**, **suggested keywords** and **practical tips**. The wider CampusConnect AI application includes registration, login, a student dashboard, job listings, profile information, application tracking, an improved-resume area and a skill-learning roadmap.

The current analyser is rule-based and keyword-driven; it does not use a trained machine-learning model or a large language model. Therefore, its score is a guidance indicator rather than a recruitment decision. The project demonstrates how a full-stack web application can automate basic resume checks and provide actionable feedback. Future improvements may include secure authentication, stronger PDF handling, job-description matching, explainable scoring, privacy controls and evaluation against a labelled dataset.

Keywords: Resume Analysis, PDF Parsing, React, Node.js, Express, MongoDB, Skill Detection, Placement Assistance, Rule-Based Evaluation.

## GRAPHICAL ABSTRACT

# CAMPUSCONNECT AI – RESUME ANALYZER




The student interacts with the React interface to upload a PDF. The Express API validates and stores the uploaded file, then pdf-parse extracts text. Rule-based checks identify resume sections and keywords. The backend calculates a weighted score and returns structured feedback to the interface. MongoDB stores student and application records through Mongoose.

*The diagram represents the architecture visible in the supplied project source. It does not imply that resume text is stored permanently in MongoDB.*

## ABBREVIATIONS

| Abbreviation | Meaning |
|---|---|
| AI | Artificial Intelligence |
| API | Application Programming Interface |
| BCA | Bachelor of Computer Applications |
| CSS | Cascading Style Sheets |
| DB | Database |
| HTTP | Hypertext Transfer Protocol |
| HTML | Hypertext Markup Language |
| JSON | JavaScript Object Notation |
| MERN | MongoDB, Express, React, Node.js (technology-family shorthand) |
| PDF | Portable Document Format |
| REST | Representational State Transfer |
| UI | User Interface |
| UID | University Identification Number |
| URL | Uniform Resource Locator |
| UX | User Experience |
| Vite | Frontend development and build tool |

## Symbols and Notation

| Symbol / Term | Meaning |
|---|---|
| S | Final resume score (0–100) |
| $w_i$ | Weight assigned to a resume criterion |
| $c_i$ | Criterion value: 1 when detected, otherwise 0 |
| Σ | Summation of weighted criteria |
| % | Percentage or normalized score indicator |
| → | Direction of data or process flow |

# CHAPTER 1. INTRODUCTION

## 1.1 Identification of Client / Need / Relevant Contemporary Issue

The primary user group for this project is students and early-career applicants who prepare resumes for internships, campus placements and entry-level jobs. Students often have academic projects, coursework, certifications and practical skills but may not know how to organize this information in a clear, recruiter-friendly format.

The need addressed by the project is practical: before submitting a resume, an applicant needs a quick way to check whether essential sections and contact details are present. A manual review can be useful, but it may take time and the quality of feedback can vary between reviewers. A structured first-level check can help a student notice omissions and improve the document before seeking detailed human feedback.

The submitted project package is branded CampusConnect AI and contains a dedicated Resume Analyzer module. The home page positions the application as a placement-support platform. It connects resume preparation with job exploration, student profile management, application tracking and skill development.

The project does not claim that all resumes can be judged by a single score. Resume quality depends on the target role, relevance of experience, clarity, accuracy, formatting and employer expectations. The current implementation focuses on detectable text and section presence, making it suitable as a preliminary checklist rather than a final hiring assessment.

## 1.2 Identification of Problem

Students may submit resumes that omit **contact details**, **a professional summary**, education information, project descriptions, relevant skills, experience, certifications, achievements or professional profile links. They may also use generic phrases and fail to describe project technologies or measurable outcomes.

The broad problem is the lack of an accessible, consistent and quick preliminary review process for student resumes. Applicants need to understand which common resume elements are present, which are missing and what improvements could make the document clearer and more relevant.

## 1.3 Identification of Tasks

- Study the existing project package and identify the frontend, backend, data models and resume-analysis logic.
- Identify user roles, functional requirements and limitations of the current implementation.
- Design the workflow for PDF upload, text extraction, content checks, scoring and feedback display.
- Implement and connect the React interface with the Express API.
- Validate file handling, readable text extraction, scoring and response generation using test cases.
- Document the system, prepare a user manual and identify future security and accuracy improvements.

### 1.4 Timeline

| Phase | Main activity | Indicative duration | Deliverable |
|---|---|---|---|
| 1 | Requirement understanding and source review | Week 1 | Requirement notes |
| 2 | UI and system design | Week 2 | Page flow and architecture |
| 3 | Frontend development | Weeks 3–4 | React pages and navigation |
| 4 | Backend and database integration | Weeks 4–5 | Express APIs and MongoDB models |
| 5 | Resume parsing and rule checks | Week 6 | Analysis response |
| 6 | Testing and refinement | Week 7 | Test log and fixes |
| 7 | Report and demonstration | Week 8 | Final report and project demo |

## 1.5 Organization of the Report

- Chapter 1: Introduces the problem, users, tasks and schedule.
- Chapter 2: Reviews resume screening concepts and existing approaches.
- Chapter 3: Describes requirements, constraints, architecture, design alternatives and implementation methodology.
- Chapter 4 :Explains the implemented modules and validation approach.
- Chapter 5 :Concludes the work and proposes future improvements. References list technical documentation, while the appendix and user manual provide checklists and operating instructions.

# CHAPTER 2. LITERATURE REVIEW / BACKGROUND STUDY

## 2.1 Timeline of the Reported Problem

Resume screening has evolved alongside recruitment practices and digital document formats. Earlier screening was commonly performed by reading printed or emailed resumes manually. As online job portals and applicant tracking systems became common, resumes were increasingly submitted in digital formats and searched using structured fields or keywords.

The growth of PDF and word-processing formats made document parsing important. A digital resume may contain text, links, columns, tables, images or scanned pages. Automated systems therefore need to extract information while accounting for differences in layout and document quality.

More recent recruitment tools use keyword matching, rule-based checks, ranking algorithms and, in some cases, machine-learning or language-processing techniques. These approaches vary in complexity, transparency, data requirements and risk of bias. The present project implements a lightweight rule-based approach that is easier to explain and suitable for a student-level prototype.

## 2.2 Existing Solutions

Existing resume-review approaches can be grouped into manual review, checklist-based review, keyword matching, applicant tracking systems and AI-assisted resume feedback platforms. Manual review can consider context but requires reviewer time. Checklists are transparent but depend on the reviewer. Keyword matching is fast but can miss equivalent wording. Advanced systems may interpret context more effectively but require careful validation and governance.

| Approach | Key feature | Advantages | Limitations |
|---|---|---|---|
| Manual review | Human reads full resume | Contextual judgement | Time-consuming; reviewer variation |
| Checklist | Checks standard sections | Simple and explainable | Does not assess depth or relevance |
| Keyword scan | Searches skills and terms | Fast; easy to implement | Synonyms and context may be missed |
| Applicant tracking system | Stores and filters applications | Useful at scale | Configuration and parsing quality matter |
| ML/NLP-based review | Uses learned patterns or language features | Can model richer signals | Needs data, validation and bias controls |
| This project | PDF text extraction + rules + weighted score | Low complexity; transparent criteria | Not semantic; not a hiring prediction |

### 2.3 Bibliometric / Feature-Based Analysis

A formal bibliometric study requires a defined publication database, search query, date range, inclusion criteria and reproducible count of selected publications. Those research records were not supplied with the project package; therefore, this report does not invent publication counts or citation statistics. Instead, a qualitative feature-based comparison is used to understand the design space.

The main dimensions considered are document compatibility, section detection, skill extraction, scoring transparency, feedback usefulness, data privacy, cost and implementation complexity. These dimensions directly inform the selection of a lightweight student-facing prototype.

## CHAPTER 2. LITERATURE REVIEW / BACKGROUND STUDY (CONTINUED)

### 2.4 Review Summary

The review indicates that an effective resume support tool should provide understandable feedback, identify missing information and avoid presenting automated output as an unquestionable judgement. A first-level analyser can be useful if it communicates exactly what it checks and makes its limitations clear.

The supplied project follows this principle in part: the backend returns separate fields for score, detected skills, sections, strengths, improvements, issues, good content, tips and suggested keywords. This structured response allows the interface to display more than a single number.

### 2.5 Problem Definition

The project aims to develop and document a web application that accepts a text-based PDF resume, extracts readable text, detects selected resume sections and technical keywords, calculates a weighted completeness-oriented score, and provides improvement guidance. The system is intended for student self-review and placement preparation.

The current scope does not include automated hiring decisions, verification of candidate claims, semantic understanding of every sentence, guaranteed ATS compatibility, image-based OCR for scanned resumes, or a validated prediction of interview selection.

### 2.6 Goals / Objectives

- Provide a simple interface for selecting and uploading a PDF resume.
- Reject unsupported file types and report unreadable or empty PDF text.
- Detect common contact details, resume sections and selected technical skills.
- Calculate a score using documented criterion weights with a maximum of 100 points.
- Generate strengths, missing-section suggestions, issues, keywords and practical tips.
- Integrate the analyser into a wider student placement-support application.
- Validate key behaviours and document known limitations and risks.

# CHAPTER 3. DESIGN FLOW / PROCESS

## 3.1 Evaluation and Selection of Specifications / Features

The feature set was selected from the needs of student applicants and from the capabilities already present in the submitted source code. The main feature is PDF resume analysis. Supporting features include student registration and login, a dashboard, profile management, job browsing, application tracking, an improved-resume area and a skill roadmap.

| ID | Requirement | Priority | Implementation evidence |
|---|---|---|---|
| FR-01 | Upload PDF resume | High | ResumeAnalyzer UI + Multer endpoint |
| FR-02 | Extract text from PDF | High | pdf-parse PDFParse.getText() |
| FR-03 | Detect contact details | High | Email and phone regular expressions |
| FR-04 | Detect resume sections | High | Keyword and phrase checks |
| FR-05 | Detect technical skills | High | Predefined skill list |
| FR-06 | Generate score and feedback | High | Weighted rules and response arrays |
| FR-07 | Register/login student | Medium | Express APIs + Student model |
| FR-08 | Track job applications | Medium | Application model and API routes |
| FR-09 | Show skill roadmap | Medium | React SkillRoadmap component |

# CHAPTER 3. DESIGN FLOW / PROCESS (CONTINUED)

## 3.2 Design Constraints

### Technical constraints

The analyser accepts PDF files only. The current parser extracts text from text-based PDFs; scanned image-only PDFs may return no readable text because no OCR stage is visible in the supplied code. The list of skills and phrases is predefined, so unlisted skills and equivalent expressions may not be detected.

### Security and privacy constraints

Resume files contain personal information. The current backend saves uploaded files in a local uploads directory. The supplied registration code compares passwords directly and stores the password field in the Student model; password hashing and token-based authentication are not implemented in the reviewed source. These are important limitations that must be addressed before real-world deployment.

### Ethical and social constraints

Automated scores can create false confidence or discourage applicants if interpreted as an objective measure of ability. The tool should explain that the score reflects the presence of programmed resume elements, not a candidate's worth or job suitability. Users should be encouraged to review suggestions and retain control over their personal data.

### Economic and operational constraints

The stack uses widely adopted open-source tools, reducing licensing costs. The system still requires a compatible runtime, a MongoDB service, local configuration, storage capacity and maintenance.

## CHAPTER 3. DESIGN FLOW / PROCESS (CONTINUED)

### 3.3 Analysis of Features and Finalization Subject to Constraints

The final feature set keeps the PDF upload, extraction, section checks, skill detection, weighted scoring and feedback generation because these features are implemented in the source and form the core value of the analyser. The broader placement modules are included as supporting functionality.

Features that require additional datasets or external services—such as live job matching, semantic comparison with job descriptions, OCR, automated rewriting using a language model and recruiter-side ranking—are not treated as implemented features. They are listed as future work.

### 3.4 Design Flow – Alternative Designs

#### Alternative A: Browser-only analysis

In this design, the browser would parse the PDF and calculate the score locally. It could reduce server transfer of resume content, but it would require a suitable client-side PDF parser and careful handling of browser memory and compatibility.

#### Alternative B: Backend-based analysis (selected)

In this design, the browser uploads the PDF to an Express API. The backend validates the file, extracts text and applies consistent analysis rules before returning a JSON result. This matches the submitted project implementation and centralizes parsing and scoring logic.

#### Alternative C: External AI service

A third design could send extracted text to an external NLP or language-model service. It may provide richer feedback but introduces API cost, privacy, consent, reliability and explainability concerns. It is outside the current implementation.

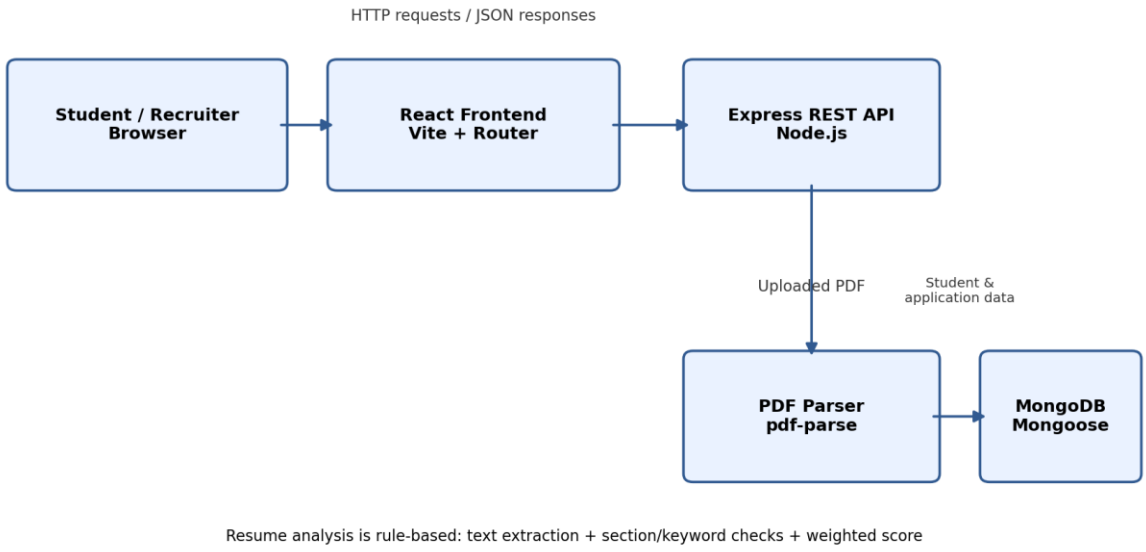
# CHAPTER 3. DESIGN FLOW / PROCESS (CONTINUED)

## 3.5 Design Selection

Alternative B was selected because it corresponds to the code provided in the project archive, supports a clear separation between interface and analysis logic, and allows the same backend rules to be applied to each uploaded PDF. It also integrates with the existing Express APIs and MongoDB-backed student and application modules.

| Selection criterion | Browser-only | Backend-based (selected) | External AI service |
|---|---|---|---|
| Matches current code | No | Yes | No |
| Central rule consistency | Moderate | High | Depends on service |
| Privacy exposure | Lower server exposure | Resume sent to own server | Potential third-party transfer |
| Implementation effort | Medium | Already implemented | High |
| Semantic feedback | Limited | Limited | Potentially richer |

## 3.6 High-Level Architecture




*Architecture is derived from the uploaded source: React frontend, Express backend, pdf-parse processing and Mongoose models connected to MongoDB.*

## CHAPTER 3. DESIGN FLOW / PROCESS (CONTINUED)

### 3.6.1 Technology Stack

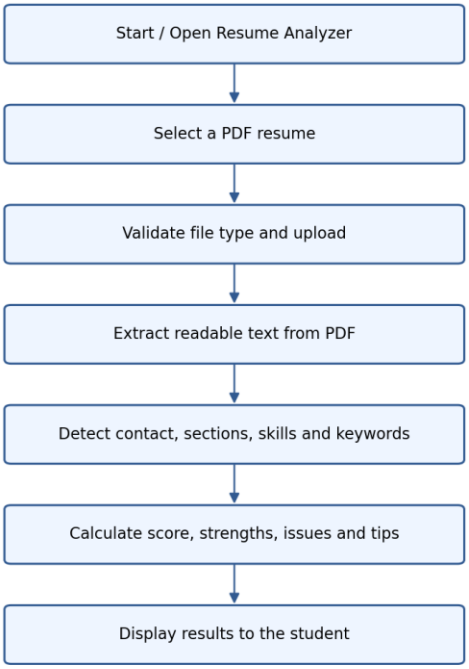
| Technology | Role in project | Reason for use |
|---|---|---|
| React 19 | Frontend component library | Reusable interactive pages and state management |
| Vite 8 | Development server and build tool | Fast frontend development workflow |
| React Router 7 | Client-side navigation | Routes between home, login, jobs and analyser pages |
| Node.js | Backend JavaScript runtime | Runs the API server |
| Express 5 | HTTP API framework | Defines registration, login, analysis and application routes |
| MongoDB | Document database | Stores student and application records |
| Mongoose 9 | MongoDB object modelling | Defines Student and Application schemas |
| Multer 2 | Multipart file upload middleware | Receives PDF resume uploads |
| pdf-parse 2 | PDF text extraction | Extracts readable text for rule checks |

### 3.6.2 Data Flow

The user selects a PDF in the Resume Analyzer page. The React component creates FormData and sends a POST request to /api/analyze-resume. Multer processes the multipart upload and stores the file. The server reads the file buffer and calls PDFParse.getText(). The extracted text is checked for contact information, headings, skill names, project evidence and measurable results. The server returns a JSON object. The frontend stores the analysis result in component state and local storage for the current browser experience, then displays the feedback.

# CHAPTER 3. DESIGN FLOW / PROCESS (CONTINUED)

## 3.6.3 Design Flow / Algorithm



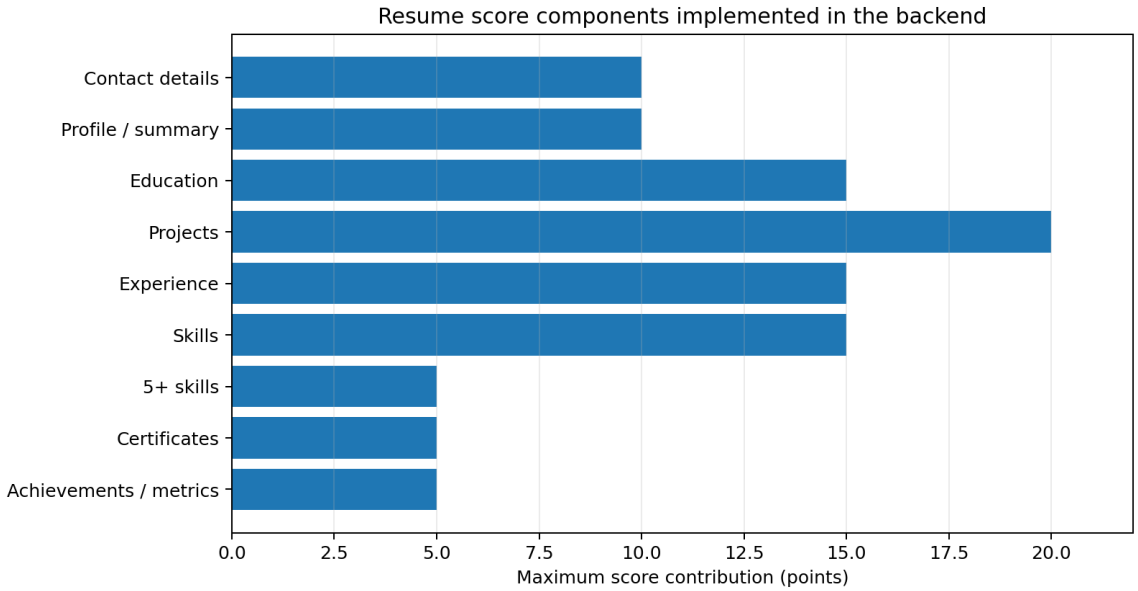

### Algorithm

- Start the application and open the Resume Analyzer module.
- Select a resume file and verify that it is a PDF.
- Submit the file to the backend using multipart form data.
- Validate the uploaded file and extract its text.
- If no readable text is found, return a clear error message.
- Convert extracted text to lowercase for case-insensitive checks.
- Detect contact details, sections, skills, project evidence and metrics.
- Apply weighted scoring rules and build feedback arrays.
- Return the analysis as JSON and render the results in the frontend.
- End the analysis process and clean up the PDF parser.

## CHAPTER 3. DESIGN FLOW / PROCESS (CONTINUED)

### 3.6.4 Resume Scoring Model

The backend uses a weighted checklist. Each criterion contributes points when its corresponding condition is detected. The maximum possible contribution is 100 points.




| Criterion | Weight | Detection condition (summary) |
|---|---|---|
| Contact information | 10 | Both email and phone detected |
| Profile / summary | 10 | Profile, summary, objective or about-me wording |
| Education | 15 | Education, academic, university, college or degree wording |
| Projects | 20 | Project-related wording |
| Experience | 15 | Experience, internship, intern or employment wording |
| Skills | 15 | Skills heading or at least one detected skill |
| Five or more skills | 5 | At least five unique predefined skills detected |
| Certifications | 5 | Certification, certificate or course wording |
| Achievements / measurable data | 5 | Achievement/award wording or numeric evidence |

Formula: $S = \min(100, \Sigma(w_i \times c_i))$, where $c_i$ is 1 when a condition is true and 0 otherwise. This is a completeness-oriented heuristic, not a statistical or machine-learning prediction.

### 3.6.5 Database Design

The supplied backend defines a Student model and an Application model using Mongoose. Student records contain full name, email, phone, course, graduation year, college, password and a tipTokens value. Application records contain a reference to the student, job title, company and status, along with timestamps.

| Collection / Model | Fields | Purpose |
|---|---|---|
| Student | fullName, email, phone, course, graduationYear, college, password, tipTokens | Student registration and profile identity |
| Application | studentId, jobTitle, company, status | Placement application tracking |

The resume-analysis response is returned to the frontend as JSON. The reviewed code also stores the analysis result in browser localStorage for the user interface. The report does not assume that full resume text is stored in MongoDB.

### API endpoints identified

| Method | Endpoint | Purpose |
|---|---|---|
| GET | / | Backend health message |
| POST | /api/register | Create student record |
| POST | /api/login | Validate login details |
| POST | /api/analyze-resume | Upload and analyse PDF |
| POST | /api/applications | Save job application |
| GET | /api/applications/:studentId | Retrieve student applications |
| DELETE | /api/applications/:id | Remove an application |
| POST | /api/resume/upload | Additional resume upload route |
| POST | /api/tip-tokens/buy | Tip-token related route |

# CHAPTER 4. RESULTS ANALYSIS AND VALIDATION

## 4.1 Implementation of Solution

The supplied project is organized as a React frontend and an Express backend. The frontend package uses React, React DOM and React Router, with Vite scripts for development, build, lint and preview. The backend package uses Express, CORS, dotenv, Mongoose, Multer and pdf-parse.

The Resume Analyzer component provides file selection, file-type validation, an analysis action, loading and error states, and result presentation. It keeps the selected file and analysis response in React state. It also uses browser localStorage for demo token counts, unlocked sections and the saved analysis response.

The backend analysis route validates the upload, reads the PDF buffer, extracts text and rejects empty text. It then performs checks for email and Indian mobile phone patterns, LinkedIn and GitHub mentions, profile headings, education, projects, experience, skills, certifications and achievements.

### 4.1.1 Main Output Fields

- score – numeric score capped at 100.
- skills – unique detected skills from a predefined list.
- sections – Boolean indicators for major resume sections and profile links.
- strengths – positive findings based on detected content.
- improvements – suggestions for missing or weak elements.
- issues – highlighted warnings and errors with categories and tips.
- goodContent – sections detected as positive content.
- tips – general practical improvement guidance.
- suggestedKeywords – predefined terms not found in the resume text.
- resumeText – extracted text truncated to the first 10,000 characters.

### 4.1.2 Frontend Modules

| Module | Purpose / observed behaviour |
|---|---|
| Home | Introduces CampusConnect AI and links users to jobs and resume analysis. |
| Register / Login | Collects student details and communicates with backend registration/login APIs. |
| Dashboard | Provides a student-oriented overview after entering the application. |
| Profile | Displays or manages student profile information. |
| Resume Analyzer | Uploads a PDF, calls the analysis API and displays feedback. |
| Jobs | Provides the job browsing interface. |
| Application Tracking | Shows saved application information and status. |
| Skill Roadmap | Shows skill learning steps and local progress tracking. |
| Improved Resume | Provides a separate area related to resume improvement. |

These modules make the analyser part of a wider placement-support workflow. The report describes only features evidenced in the source files and does not claim integration with external job boards or employer databases.

### 4.1.3 Result Interpretation

A high score means that more of the programmed checklist conditions were detected. It does not prove that the resume is factually correct, well written, relevant to a particular vacancy or likely to be shortlisted. A low score may reflect missing headings or wording that the rules do not recognize, even when the candidate has relevant experience.

### 4.1.4 Validation Strategy

Validation should cover the upload boundary, parser response, rule conditions, score calculation and frontend display. Test inputs should be synthetic or explicitly authorized sample resumes. Personal resumes should not be used in demonstrations without permission, and uploaded files should be removed after testing where retention is not required.

| Test ID | Input / condition | Expected behaviour |
|---|---|---|
| TC-01 | No file selected | Frontend asks user to upload a resume |
| TC-02 | Non-PDF file | Frontend or backend rejects the file |
| TC-03 | Valid text-based PDF | Text is extracted and analysis JSON is returned |
| TC-04 | Scanned/image-only PDF | Readable-text error is returned if extraction is empty |
| TC-05 | Email and phone both present | Contact criterion contributes 10 points |
| TC-06 | Only email or only phone present | Contact criterion does not receive its 10 points |
| TC-07 | Five or more listed skills | Additional 5-point skill-count criterion is applied |
| TC-08 | Project heading absent | Project improvement and issue feedback is generated |
| TC-09 | Resume with all detectable criteria | Score is capped at 100 |
| TC-10 | Backend unavailable | Frontend displays an analysis failure message |

*These are validation cases derived from the implementation logic. They are not presented as executed test results; record actual pass/fail outcomes during your project demonstration.*

### 4.1.5 Expected Result Screens / Evidence

The following screenshots should be inserted from the running application before final submission. Use genuine screenshots from the project rather than illustrative images.

- Figure 4.1 – CampusConnect AI home page and navigation.
- Figure 4.2 – Student registration or login screen.
- Figure 4.3 – Resume Analyzer page before selecting a file.
- Figure 4.4 – Selected PDF file and analysis action.
- Figure 4.5 – Resume score and detected skills.
- Figure 4.6 – Strengths, issues, improvements and suggested keywords.
- Figure 4.7 – Application tracking or skill roadmap module.

### 4.1.6 Analysis of Results

The rule-based output is explainable because each score component is linked to a visible condition in the backend. For example, contact information receives points only when both an email address and a phone number are detected. A project section can be detected through project-related terms, while project quality checks look for terms associated with technologies and outcomes.

However, keyword checks can produce false positives when a word appears in an unrelated context and false negatives when a resume uses a synonym or a non-standard heading. The score should therefore be treated as a checklist summary and not as an objective ranking of candidates.

### 4.1.7 Limitations, Risks and Mitigation

| Limitation / risk | Effect | Recommended mitigation |
|---|---|---|
| Rule-based phrase detection | May miss synonyms or context | Use configurable synonym dictionaries and NLP evaluation |
| PDF text extraction only | Scanned PDFs may fail | Add OCR with user consent and quality checks |
| Fixed skill list | New or uncommon skills may be missed | Maintain a reviewed, role-specific skill taxonomy |
| Heuristic score | May be misunderstood as employability score | Show scoring explanation and clear disclaimer |
| Plaintext password handling in current source | Account compromise risk | Hash passwords with a modern password-hashing algorithm |
| Local upload storage | Personal data may remain on server | Add access controls, retention policy and secure deletion |
| No visible authorization middleware | API access may be insufficiently restricted | Add sessions or signed tokens and ownership checks |
| Client-side localStorage | Data can persist on shared devices | Provide clear-data/logout controls and avoid sensitive storage |

Privacy and security are essential because resumes contain contact details, education and work history. Before deployment beyond a controlled academic demonstration, the project should implement secure authentication, file-size limits, upload scanning, access controls, encrypted transport, retention rules and a privacy notice.

# CHAPTER 5. CONCLUSION AND FUTURE WORK

## 5.1 Conclusion

The Resume Analyzer project demonstrates a full-stack web application that helps students perform a preliminary review of a resume. The submitted source combines a React-based user interface with a Node.js and Express backend. It accepts PDF files, extracts readable text through pdf-parse and evaluates content using regular expressions, predefined keyword lists and section-presence rules.

The analyser produces a weighted score and multiple feedback categories rather than returning only a pass/fail result. This can help students notice missing contact information, profile summaries, education, projects, experience, technical skills, certifications, achievements and professional links. The wider CampusConnect AI modules provide a placement-oriented context through student profiles, job browsing, application tracking and skill roadmaps.

The project meets its educational objective of demonstrating frontend-backend communication, file upload handling, PDF text extraction, rule-based analysis, REST API design and database modelling. The current score is a heuristic measure of detectable resume completeness. It is not a trained AI prediction, ATS guarantee or hiring recommendation.

A final evaluation should include real execution evidence, recorded test results and supervisor review. Any differences between expected and observed behaviour should be documented honestly, particularly for scanned PDFs, unusual formatting, missing keywords and backend configuration.

## CHAPTER 5. CONCLUSION AND FUTURE WORK (CONTINUED)

### 5.2 Future Work

- Secure authentication: hash passwords, add session or token-based authentication, validate authorization and protect student-specific APIs.
- Improved document processing: support DOCX files, OCR for scanned PDFs, multi-column layouts and extraction confidence indicators.
- Job-description matching: allow users to paste a vacancy description and compare required skills with detected resume content.
- Context-aware analysis: use NLP to identify action verbs, project responsibilities, measurable outcomes and duplicated statements.
- Explainable scoring: show each criterion, its weight, evidence detected and reason for missing points.
- Role-specific skill libraries: maintain separate skill sets for data analyst, software developer, designer and other job families.

These extensions should be prioritized according to user need, available time, privacy impact and measurable improvement in analysis quality.

## REFERENCES


[1] React Documentation, “Quick Start,” https://react.dev/learn (accessed 30 October 2026).

[2] Vite Documentation, “Getting Started,” https://vite.dev/guide/ (accessed 30 October 2026).

[3] Node.js Documentation, https://nodejs.org/docs/ (accessed 30 October 2026).

[4] Express Documentation, “Express 5.x,” https://expressjs.com/ (accessed 30 October 2026).

[5] MongoDB Documentation, https://www.mongodb.com/docs/ (accessed 30 October 2026).

[6] Mongoose Documentation, https://mongoosejs.com/docs/ (accessed 30 October 2026).

[7] Multer Documentation, https://github.com/expressjs/multer (accessed 30 October 2026).

[8] pdf-parse package documentation, https://www.npmjs.com/package/pdf-parse (accessed 30 October 2026).

[9] MDN Web Docs, “Using Fetch,” https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch (accessed 30 October 2026).

[10] MDN Web Docs, “Web Storage API,” https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API (accessed 30 October 2026).

[11] OWASP Foundation, “Password Storage Cheat Sheet,” https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (accessed 30 October 2026).

[12] OWASP Foundation, “File Upload Cheat Sheet,” https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html (accessed 30 October 2026).

[13] Uploaded source project archive, CampusPlacementAI – Copy.zip, supplied by the project team.

[14] Chandigarh University, Project Report Format, 18-page template supplied by the project team.


*Verify URLs and access dates at the time of final submission. Follow the citation style required by your department if it differs.*

# APPENDIX

## Appendix A – Design and Submission Checklist

- Confirm project title, student name(s), UID(s), programme, department and academic year.
- Obtain supervisor and Head of Department approval for the certificate page.
- Run the frontend and backend using the correct environment configuration.
- Confirm MongoDB is available and MONGO_URI is configured in the backend environment.
- Capture genuine screenshots of the running application and insert them in Chapter 4.
- Run each test case and record actual Pass / Fail status with date and remarks.
- Do not include real student resumes or personal data in the report without permission.
- Review password storage, upload access, file retention and API authorization before any public deployment.
- Check spelling, page numbering, table of contents and figure numbering.
- Attach the institution-required plagiarism report separately if required.

## Appendix B – Sample Test Record

| Test ID | Actual result | Status | Remarks / evidence |
|---|---|---|---|
| TC-01 | _____ | Pass / Fail | _____ |
| TC-02 | _____ | Pass / Fail | _____ |
| TC-03 | _____ | Pass / Fail | _____ |
| TC-04 | _____ | Pass / Fail | _____ |
| TC-05 | _____ | Pass / Fail | _____ |
| TC-06 | _____ | Pass / Fail | _____ |
| TC-07 | _____ | Pass / Fail | _____ |

## Appendix C – Standards and Good Practices

The supplied university template marks the standards list as mandatory for engineering programmes. For this BCA report, a standards list is not asserted as mandatory unless the department instructs otherwise. Relevant good practices include secure password storage, safe file upload validation, privacy-by-design, accessible web interfaces and responsible use of automated decision-support tools.

## USER MANUAL

### 1. System Requirements

- A desktop or laptop with a modern web browser.
- Node.js installed for running the frontend and backend.
- MongoDB service or an accessible MongoDB database.
- Project source folder with frontend and server dependencies installed.
- A readable, text-based PDF resume for analysis.

### 2. Configure the Backend

Open the project folder in Visual Studio Code. Open a terminal and move into the server directory. Install dependencies if they are not already installed. Create a .env file in the server folder and define MONGO_URI with the connection string for the MongoDB database. Confirm that the database service is running.

Run the backend using the project's server entry point (for example, node server.js from the server folder). The source sets the API port to 5000. The root endpoint can be opened in a browser to confirm that the backend returns its running message.

### 3. Start the Frontend

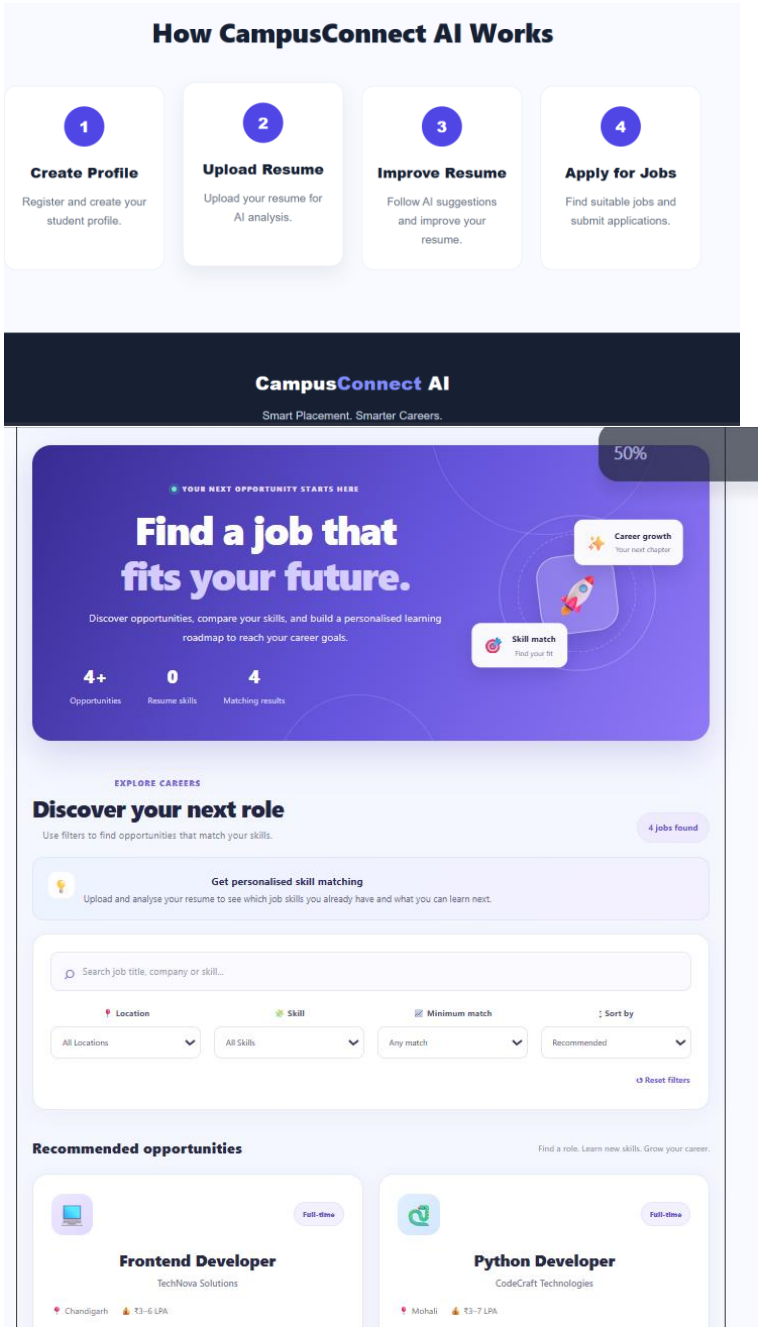
Open a second terminal in the frontend project root. Install dependencies if needed, then run npm run dev. Vite displays a local development URL in the terminal. Open that URL in a browser. If the frontend cannot reach the API, confirm that the backend is running on port 5000 and that the API URL in ResumeAnalyzer.jsx matches the backend address.

### 4. Analyse a Resume

- Open the CampusConnect AI website.
- Choose Resume Analyzer from the navigation or home-page action.
- Select a PDF resume from the computer.
- Check that the selected file is the intended version and contains selectable text.
- Click the analysis action and wait for the request to complete.
- Review the score, detected skills, section indicators, strengths, issues, improvements, suggested keywords and tips.
- Use the feedback as a checklist and manually verify every recommendation before editing the resume.

## 5. Troubleshooting and Safe Use

- If the file is rejected, confirm that it is a PDF and not a renamed DOCX file.
- If the parser reports no readable text, export the resume as a text-based PDF rather than a scanned image.
- If analysis fails, check that the backend is running and MongoDB configuration is valid.
- If registration or login fails, verify required fields and database connectivity.
- Do not upload confidential resumes on a shared or public computer.
- Do not treat the score as a guarantee of interview selection or job suitability.


How CampusConnect AI Works
1
Create Profile
Register and create your student profile.
2
Upload Resume
Upload your resume for AI analysis.
3
Improve Resume
Follow AI suggestions and improve your resume.
4
Apply for Jobs
Find suitable jobs and submit applications.
CampusConnect AI
Smart Placement. Smarter Careers.
50%
YOUR NEXT OPPORTUNITY STARTS HERE
Find a job that fits your future.
Discover opportunities, compare your skills, and build a personalised learning roadmap to reach your career goals.
4+
Opportunities
0
Resume skills
4
Matching results
Career growth
Skill match
EXPLORE CAREERS
Discover your next role
Use filters to find opportunities that match your skills.
4 jobs found
Get personalised skill matching
Upload and analyse your resume to see which job skills you already have and what you can learn next.
Search job title, company or skill...
Location
Skill
Minimum match
Sort by
All Locations
All Skills
Any match
Recommended
Reset filters
Recommended opportunities
Find a role. Learn new skills. Grow your career.
Full-time
Frontend Developer
TechNova Solutions
Chandigarh
₹3–6 LPA
Full-time
Python Developer
CodeCraft Technologies
Mohali
₹3–7 LPA